



DSA 4030: BIG DATA SECURITY

Practical Group Project

Group 2 — Insider Threat Detection

Final Consulting Report

Fortune Shammar	669891
Hana Gashaw	670555
Melissa Omondi	669782

Scenario: Detecting employee access to customer data outside working hours

Prepared for: DSA 4030 Course Instructor

Prepared by: Group 2

United States International University — Africa

Table of Contents

Table of Contents.....	2
1. Executive Summary.....	3
2. System Architecture	3
2.1 Component Summary	4
3. Implementation Report	5
3.1 Environment Setup	5
3.2 Dataset Design	5
3.3 Security Controls Implemented	5
3.4 Detection Engine Configuration	5
3.5 Dashboard Configuration.....	6
4. Security Testing Matrix	7
4.1 Root-Cause Investigation of Recall Value.....	8
5. Evidence Portfolio	8
6. Risk Assessment Table	18
7. Conclusion and Recommendations.....	18
7.1 Vulnerabilities Identified	18
7.2 Remaining Risks.....	19
7.3 Lessons Learned	19
7.4 Improvements That Should Be Implemented	19
8. Appendices	20
Appendix A: Scripts Included	20
Appendix B: Dataset Schema	20
Appendix C: Tool Versions and References.....	20

1. Executive Summary

This report presents the work of Group 2 in our role as an external cybersecurity consulting team engaged to assess and improve the security of a client organization's big data environment. The client's stated concern was that employees may be accessing customer records outside of normal working hours. Building upon the project's focus on off-hours access, we expanded the detection capability to include three independent behavioral indicators—off-hours access, abnormal access volume, and access outside an employee's assigned customer segment. We also incorporated multi-signal correlation, where the presence of multiple suspicious behaviors for the same employee results in a higher-severity alert.

We designed and built a complete detection pipeline using Python for data generation and detection logic, and OpenSearch together with OpenSearch Dashboards as the storage, query, security analytics, and visualization layer, running inside Docker containers for reproducibility. A synthetic dataset of 100,000 access events was generated, with 5,000 events deliberately constructed to represent four categories of insider-threat behavior. Our detection logic was tested against this dataset using seven automated tests, including a full precision/recall validation against the dataset's ground-truth labels.

The final, reproducible result of our testing achieved 100% precision and 90.20% recall: every alert our system generated corresponded to a genuinely planted suspicious event, and just over 90% of all planted suspicious events were successfully detected. The remaining gap was investigated in depth and traced to a specific, well-understood limitation in our bulk-volume detection rule, which is discussed fully in Section 4 (Testing) and Section 7 (Recommendations).

Our solution also includes a live monitoring dashboard summarizing alert severity, detection reasons, and top flagged employees, giving a security analyst a practical starting point for investigation. This report documents our environment setup, dataset design, security controls, detection logic, testing methodology, identified risks, and recommendations for improving the system further before any real-world deployment.

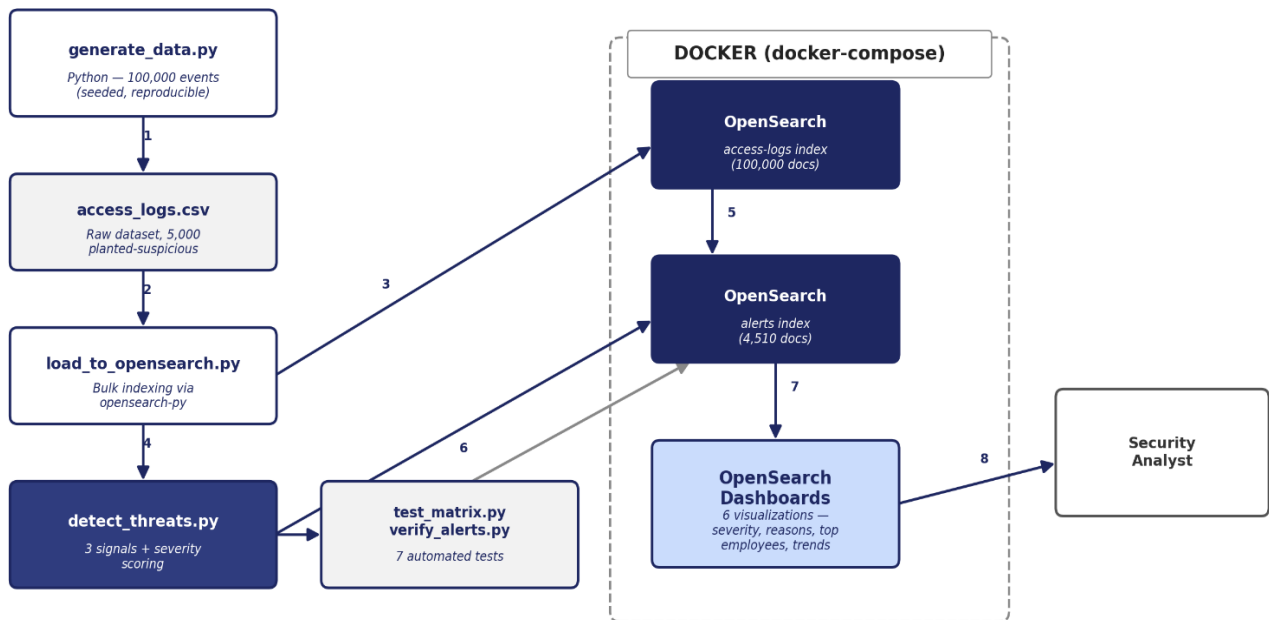
2. System Architecture

The diagram below shows the complete flow of data through our system, from generation through to analyst-facing dashboards.

The data flows sequentially through these phases:

- ☐ **Data Generation**
- ☐ **Ingestion**
- ☐ **Detection & Validation**
- ☐ **Analysis**

Insider Threat Detection — System Architecture



2.1 Component Summary

Data source: generate_data.py produces a 100,000-record synthetic access log (access_logs.csv), simulating 200 employees across four customer segments, with a fixed random seed for full reproducibility.

Storage platform: OpenSearch, running in a Docker container backed by a persistent volume, stores two indices ,access-logs (raw activity) and alerts (detection findings).

Security tooling: OpenSearch serves as the detection and analytics engine; OpenSearch Dashboards provides the analyst-facing visualization layer. Together they form a lightweight, custom-built security monitoring system, following the same architectural pattern used by many organizations that build detection capability on open-source search engines rather than a commercial SIEM.

Detection logic: detect_threats.py, written in Python, queries access-logs, applies three behavioral rules, computes a severity score, and writes structured alert records to the alerts index.

Alert Verification: queries the alerts index and the access-logs index's is_suspicious labels, then checks each alert against the true label to sort results into true positives and false positives, and separately finds any planted-suspicious events that got no alert at all (false negatives).

Testing: test_matrix.py and verify_alerts.py independently validate detection behavior against known test cases and the dataset's ground-truth labels.

Containerization: Docker Compose runs OpenSearch and OpenSearch Dashboards as two networked containers, using a named volume (opensearch-data) so that all indexed data persists across container and machine restarts.

3. Implementation Report

3.1 Environment Setup

The system was implemented using Python 3.12 and Docker. A Docker Compose file defines two services: `opensearch-node1` (the OpenSearch engine, exposed on port 9200) and `opensearch-dashboards` (the visualization layer, exposed on port 5601). Both are attached to a persistent named volume, `opensearch-data`, ensuring that all indexed documents survive container restarts and machine shutdowns without any data loss. This was verified directly: after a full laptop shutdown and restart, both indices were confirmed to still contain their expected document counts before continuing work.

3.2 Dataset Design

The dataset was generated with Python, using the `zoneinfo` library to ensure timestamps correctly reflect the client's Kenyan operating context. Containing:

- ☐ Four employee roles
- ☐ Four customer segments
- ☐ June 2026 timestamps

Two hundred simulated employees were created, each assigned a role, a home customer segment, and two typical source IP addresses. Of these, 95,000 events represent ordinary business activity, and 5,000 events were deliberately constructed across four groups to represent distinct insider-threat patterns:

- Off-hours group (10 employees, 500 events): access occurring outside 8am–6pm, from the employee's normal segment and IP.
- Bulk-volume group (10 employees, 3,000 events): 300 events per employee compressed into a single ~10-minute burst, simulating rapid data exfiltration.
- Scope-violation group (10 employees, 500 events): normal-hours access, but to a customer segment outside the employee's assigned one.
- Multi-signal group (5 employees, 1,000 events): all three anomalies combined, plus access from an unfamiliar IP address, representing the most severe simulated case.

The generator is designed with a fixed random seed (`random.seed(42)`) so that the final dataset is exactly reproducible on every run.

3.3 Security Controls Implemented

Logging: To support monitoring, auditing, and forensic investigations, the system records every simulated employee access event and every generated alert in OpenSearch. Separate indices (`access-logs` and `alerts`) preserve both the original activity logs and the corresponding detection results, creating a complete audit trail that enables investigators to reconstruct events, verify alerts, and analyze suspicious behaviour.

Behavioural Anomaly Detection: This is the primary security control implemented in the project. Three independent behavioural rules analyse employee access patterns to identify potential insider threats, including access outside working hours, unusually high access volumes, and access to customer segments outside an employee's assigned responsibility. By monitoring behaviour rather than relying solely on predefined signatures, the system can identify activities that may indicate malicious or unauthorized actions.

Alerting: Every event that satisfies one or more detection rules generates a structured alert containing a unique alert ID, the responsible employee, the triggering reason(s), and a computed severity level. These alerts provide security analysts with timely notification of suspicious activity, enabling faster investigation, incident response, and prioritization of high-risk events.

3.4 Detection Engine Configuration

`detect_threats.py` implements three signals, applied per employee:

- Off-hours access: the event's hour falls before 8am or at/after 6pm.

- Bulk volume: 50 or more of the same employee's events fall within a forward-looking rolling 10-minute window.
- Scope violation: the accessed customer segment differs from the employee's assigned segment.

Each event that trips one or more rules is scored: one signal is scored Low severity, two signals Medium, and three signals High ; allowing multi-signal cases (our most severe simulated group) to be automatically prioritized above single-signal cases.

3.4 OpenSearch Configuration

Two indices were created to separate raw activity from analytical findings:

Index	Purpose
access-logs	Stores all 100,000 simulated employee access events; the raw audit trail
alerts	Stores structured alert records generated by detection logic; each includes the responsible employee, triggering reason(s), and severity score

3.5 Dashboard Configuration

Six visualizations were built in OpenSearch Dashboards, combined into a single saved dashboard ("Insider Threat Detection Dashboard"):

- Alerts by Severity
- Alerts by Detection Reason
- Top Flagged Employees
- Alerts Over Time
- Total Access Events Over Time
- Events by Action Type.

These give an analyst an immediate, visual summary of detection findings.

3.6 Tool Selection Justification

Each tool in this project was chosen for the following reasons:

Python: Python was used to generate the synthetic dataset, implement the detection engine, generate alerts, and perform automated testing. It provided the flexibility to implement custom behavioural detection rules, including the rolling-window bulk-volume analysis, while keeping the detection logic transparent, easily auditable, and straightforward to validate. Its extensive standard library and rapid development capabilities also enabled the system to be implemented efficiently within the project timeframe.

OpenSearch: chosen as the storage and analytics engine, it was one of the explicitly suggested tools for this scenario, and because it is a genuine, industry-used foundation for building custom security monitoring; the same pattern many organizations follow when building detection capability without a commercial SIEM license. It provided an integrated environment for monitoring alerts, analysing trends, and presenting findings through interactive dashboards, fulfilling the project's requirement to summarize security events visually.

OpenSearch Dashboards: paired naturally with OpenSearch as its native visualization layer, requiring no additional connector, and providing the analyst-facing monitoring view.

Docker Compose: used to run OpenSearch and OpenSearch Dashboards in isolated, reproducible containers rather than installing them directly on the host machine. This avoided manual, error-prone installation steps, kept the environment fully isolated from the rest of the system, and, critically, allowed the exact same environment to be recreated with a single command after every restart, which was verified directly across multiple working sessions.

Detection Engine Implementation Approach: A custom Python-based detection engine was implemented instead of OpenSearch's built-in Security Analytics plugin. This approach provided greater flexibility for implementing project-specific behavioural rules, particularly the rolling-window bulk-volume detection logic, while keeping the detection process transparent, easily auditable, and straightforward to validate during testing and investigation.

4. Security Testing Matrix

Seven tests were performed. All results below were produced by `test_matrix.py`, run against the final, seeded dataset, and are fully reproducible. Each test's specific evidence file is listed in its row below; the complete text files (full terminal output) and screenshots are available in the evidence `07_testing` folder, and screenshots are additionally retained in the Evidence Portfolio (Section 5).

Test 1: Normal behaviour → no alert				
Objective	Procedure	Expected Result	Actual Result	Evidence
Confirm that ordinary employee activity does not trigger any alert.	Query the alerts index for a control employee (EMP0100) outside all planted-suspicious groups.	0 alerts for EMP0100.	0 alerts found. PASS.	test_matrix_output.txt + screenshot

Test 2: Off-hours access → alert fires				
Objective	Procedure	Expected Result	Actual Result	Evidence
Confirm off-hours access is correctly detected.	Query alerts for EMP0001 (planted off-hours group).	Alert(s) with reason off_hours_access.	50 of 50 alerts matched. PASS.	test_matrix_output.txt + screenshot

Test 3: Bulk volume → alert fires				
Objective	Procedure	Expected Result	Actual Result	Evidence
Confirm high-volume access clustering is detected.	Query alerts for EMP0011 (planted bulk-volume group).	Alert(s) with reason bulk_volume.	251 of 251 alerts matched. PASS.	test_matrix_output.txt + screenshot

Test 4: Out-of-segment access → alert fires				
Objective	Procedure	Expected Result	Actual Result	Evidence
Confirm scope violations are detected.	Query alerts for EMP0021 (planted scope-violation group).	Alert(s) with reason scope_violation.	50 of 50 alerts matched. PASS.	test_matrix_output.txt + screenshot

Test 5: Multi-signal → higher severity				
Objective	Procedure	Expected Result	Actual Result	Evidence
Confirm combined signals escalate severity.	Query alerts for EMP0031 (planted multi-signal group), inspect severity distribution.	Some alerts reach High severity.	151 High, 49 Medium. PASS.	test_matrix_output.txt + screenshot

Test 6: Alert data completeness				
Objective	Procedure	Expected Result	Actual Result	Evidence

Confirm alert records contain all fields an investigator needs.	Inspect a sample alert record's fields.	alert_id, employee_id, event_id, timestamp, reasons, severity all present.	All fields present and populated. PASS.	test_matrix_output.txt + screenshot
---	---	--	---	-------------------------------------

Test 7: Precision / Recall (full dataset)				
Objective	Procedure	Expected Result	Actual Result	Evidence
Measure overall detection accuracy against ground truth.	Compare all 4,510 generated alerts against the is_suspicious label across all 100,000 records.	Recall $\geq 90\%$, Precision $\geq 95\%$.	Precision 100.00%, Recall 90.20% (4,510 TP, 0 FP, 490 FN). PASS.	test_matrix_output.txt + investigate_fp_fn_output.txt + screenshot

4.1 Root-Cause Investigation of Recall Value

Beyond the seven pass/fail tests, we conducted a deeper root-cause investigation (investigate_fp_fn.py) into the 490 false negatives to understand precisely why they were missed.

Investigation confirmed that all sampled missed events had very high 'backward' neighbor counts (250+ suspicious events in the prior 10 minutes) despite low 'forward' counts (as few as 39–48). This proves these events are genuinely part of dense, real suspicious bursts; the detection rule's forward-only window structure, not any flaw in the underlying data or labeling, is what causes them to fall just outside the 50-event threshold near the tail end of each burst. This is a well-understood, fixable rule-design limitation, addressed further in Section 7 (Recommendations).

5. Evidence Portfolio

A complete evidence portfolio is submitted alongside this report, organized by project phase into seven folders (01_environment through 07_testing). Every screenshot referenced below is embedded directly in this section; every supporting script, log, and configuration file is listed by exact filename in its corresponding table, so that the physical file can be located immediately in the submitted evidence folder.

5.1 Project Folder Structure

The diagram below mirrors the exact folder layout submitted alongside this report, so that every file referenced throughout this document can be located quickly. Detailed contents of each evidence subfolder are listed in Sections 5.2–5.8 below.

```

Group2-InsiderThreat/
├── docker-compose.yml
├── scripts/                                (working copies used to run the project)
│   ├── generate_data.py
│   ├── load_to_opensearch.py
│   ├── detect_threats.py
│   ├── verify_alerts.py
│   ├── test_matrix.py
│   ├── investigate_fp_fn.py
│   └── access_logs.csv
├── reports/                               (final pdf report)
└── evidence/                             (submission evidence, organized by phase)
    ├── 01_environment/
    ├── 02_dataset/
    ├── 03_loading/
    ├── 04_detection/
    ├── 05_alerting/
    ├── 06_dashboard/
    └── 07_testing/

```

5.2 01_environment : Environment Setup

Filename	Description
docker-compose.yml	Defines the opensearch-node1 and opensearch-dashboards services and the persistent opensearch-data volume.
docker ps output.txt	Terminal output of docker ps confirming both containers are running.
opensearch check.txt	Terminal output confirming OpenSearch responds on port 9200.

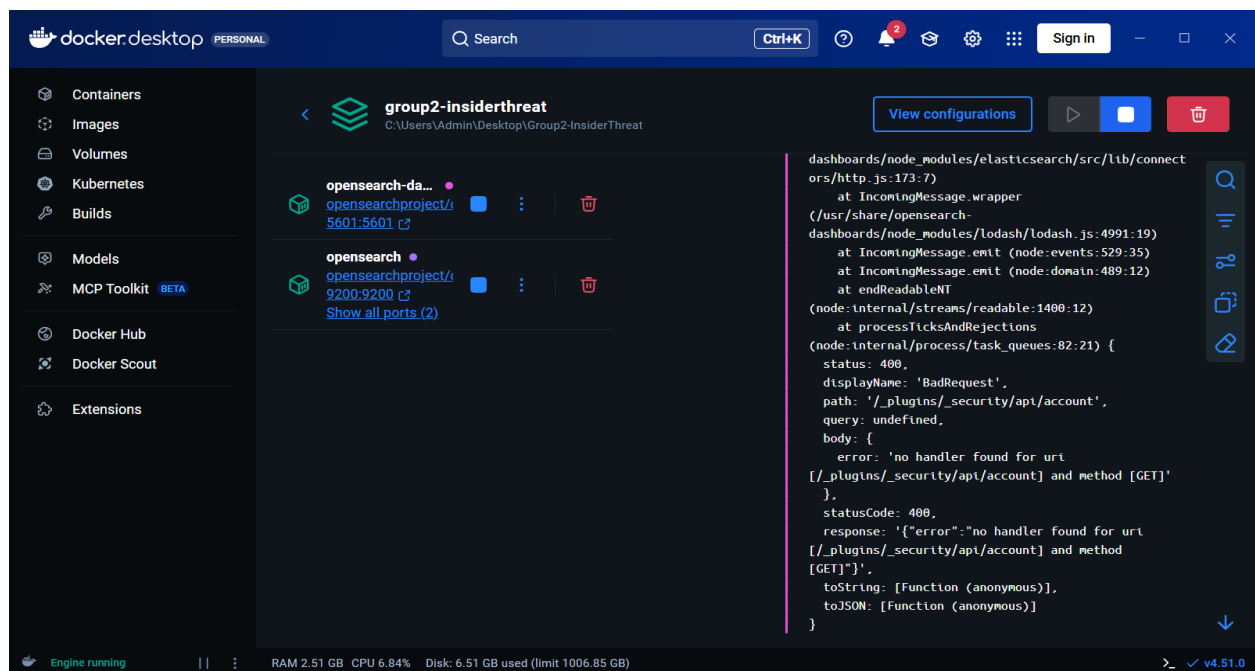


Figure 5.2a — docker_containers.png: Docker Desktop showing both containers (opensearch, opensearch-dashboards) running.

```

PS C:\Users\Admin\Desktop\Group2-InsiderThreat> docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STATUS
PORTS
NAMES
02fa7813ddee   opensearchproject/opensearch-dashboards:2.15.0   "./opensearch-dashbo..." 5 minutes ago   Up 4 minutes
0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp         opensearch-dashboards
5f678840eb00   opensearchproject/opensearch:2.15.0             "./opensearch-docker..." 5 minutes ago   Up 4 minutes
0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp, 0.0.0.0:9600->9600/tcp, [::]:9600->9600/tcp   opensearch-node1

```

Figure 5.2b — `docker_ps_screenshot.png`: terminal confirmation of both containers' status.

```

PS C:\Users\Admin\Desktop\Group2-InsiderThreat> curl http://localhost:9200

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is
parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): Y

StatusCode      : 200
StatusDescription : OK
Content         : {
  "name" : "5f678840eb00",
  "cluster_name" : "docker-cluster",
  "cluster_uuid" : "MMQts0UNSoK-hN7K-fVrUdQ",
  "version" : {
    "distribution" : "opensearch",
    "number" : "2.15.0",
    "build_ty...
RawContent      : HTTP/1.1 200 OK
                  Content-Length: 568
                  Content-Type: application/json; charset=UTF-8

                  {
                    "name" : "5f678840eb00",
                    "cluster_name" : "docker-cluster",
                    "cluster_uuid" : "MMQts0UNSoK-hN7K-fVrUdQ",
                    ...
Forms           : {}
Headers         : {[Content-Length, 568], [Content-Type, application/json; charset=UTF-8]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 568

```

Figure 5.2c — `opensearch_check.png`: OpenSearch cluster health response.

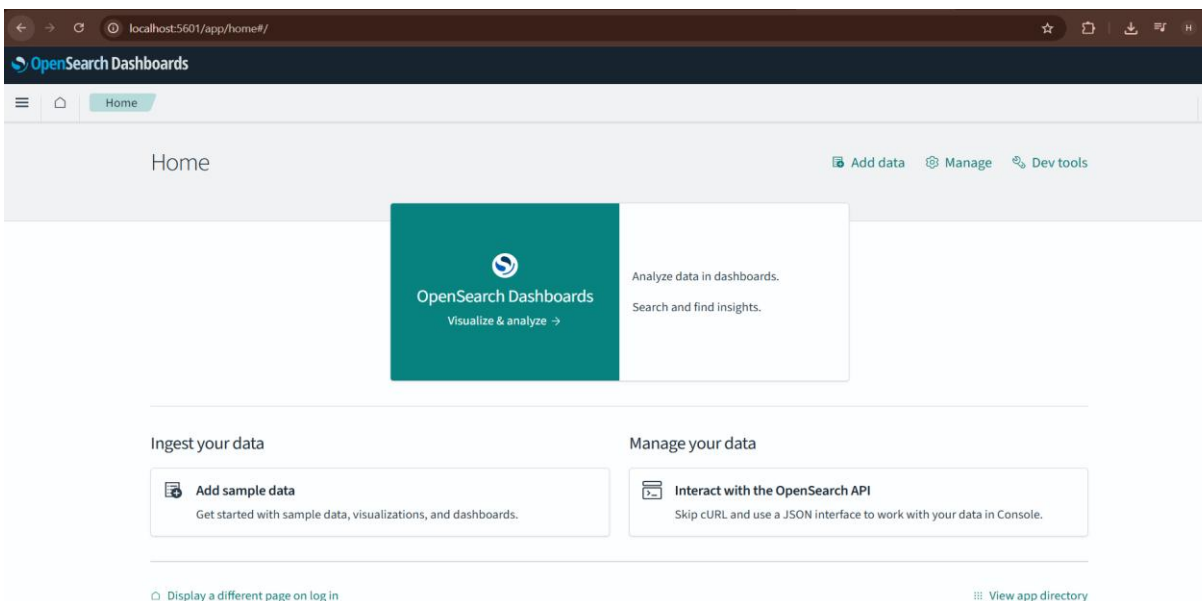
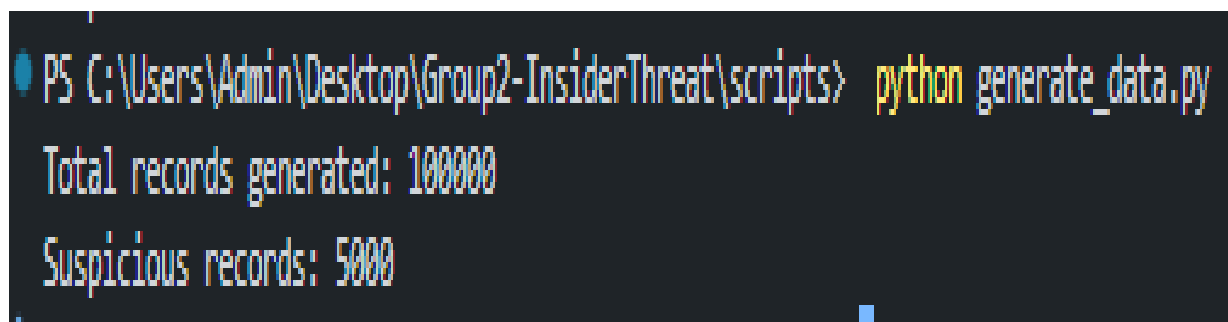


Figure 5.2d — `opensearch_dashboards_ui_loaded.png`: OpenSearch Dashboards home page confirming the UI is reachable.

5.3 02_dataset : Dataset Generation

Filename	Description
generate_data.py	Final, seeded dataset generator (random.seed(42))
access_logs.csv	The generated dataset — 100,000 records, 5,000 planted-suspicious.
generate_data_output.txt	Terminal output confirming 100,000 total records and 5,000 suspicious records generated.



```

PS C:\Users\Admin\Desktop\Group2-InsiderThreat\scripts> python generate_data.py

Total records generated: 100000

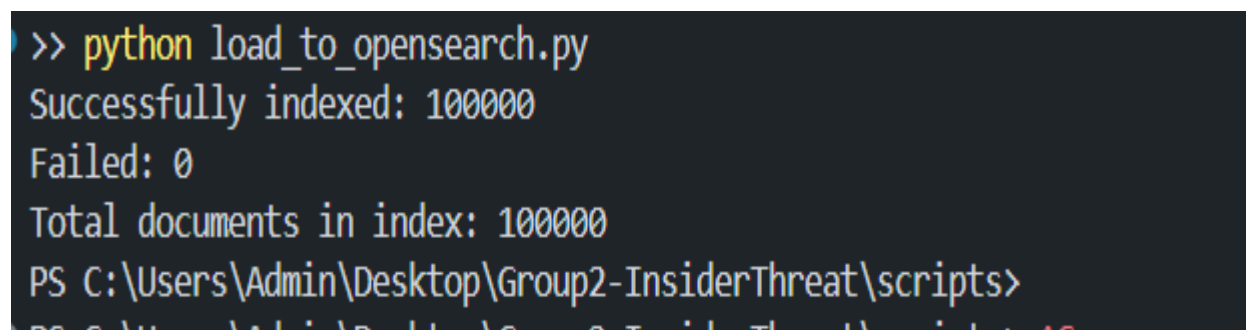
Suspicious records: 5000

```

Figure 5.3a — data_generation.png: terminal output of the final dataset generation run.

5.4 03_loading : Loading into OpenSearch

Filename	Description
load_to_opensearch.py	Bulk loader script — deletes/recreates the access-logs index with an explicit field mapping, then bulk-indexes the CSV.
loader_output.txt	Terminal output: 100,000 indexed, 0 failed.
count_verification.txt	Independent _count API check confirming 100,000 documents in access-logs.
search_sample_verification.txt	_search API check (with track_total_hits) confirming an accurate total and showing sample documents.



```

>> python load_to_opensearch.py
Successfully indexed: 100000
Failed: 0
Total documents in index: 100000
PS C:\Users\Admin\Desktop\Group2-InsiderThreat\scripts>

```

Figure 5.4a — loading.png: terminal output of the loader run.

5.5 04_detection — Detection Logic

Filename	Description
detect_threats.py	Detection logic — applies the three signals, computes severity, and writes alert records.
verify_alerts.py	Ground-truth validation script — compares generated alerts against the is_suspicious label.
detection_output.txt	Terminal output: 4,510 alerts generated and indexed.
verify_alerts_output.txt	Terminal output: 4,510 true positives, 0 false positives, 490 false negatives.
detection_limitations_note.txt	Written explanation of the forward-only rolling window limitation and the dataset fix that eliminated false positives.

```

PS C:\Users\Admin\Desktop\Group2-InsiderThreat\scripts> python detect_threats.py
Fetching records from OpenSearch...
Fetched 100000 records from 'access-logs'
Total alerts to index: 4510
Indexed alerts: 4510
Failed: 0
Total documents in alerts index: 4510
PS C:\Users\Admin\Desktop\Group2-InsiderThreat\scripts>

```

Figure 5.5a — *detect_threats_run_screenshot.png*: terminal output of the final detection run.

5.6 05_alerting — Alert Generation

Alerting is implemented within `detect_threats.py` itself (Section 3.3); this folder contains the supporting evidence specific to the alerting requirement.

Filename	Description
<code>detect_threats.py</code>	Same detection/alerting script, retained here to evidence the alert-writing logic specifically.
<code>detection_output.txt</code>	Same run output, retained here as alerting evidence.
<code>sample_alerts.txt</code>	Three real alert records retrieved directly from the alerts index, showing the fields available to an investigator.
<code>alerting_note.txt</code>	Short design note explaining why alerting was implemented inside the detection script rather than as a separate step.

5.7 06_dashboard — Dashboard

All nine files in this folder are screenshots;

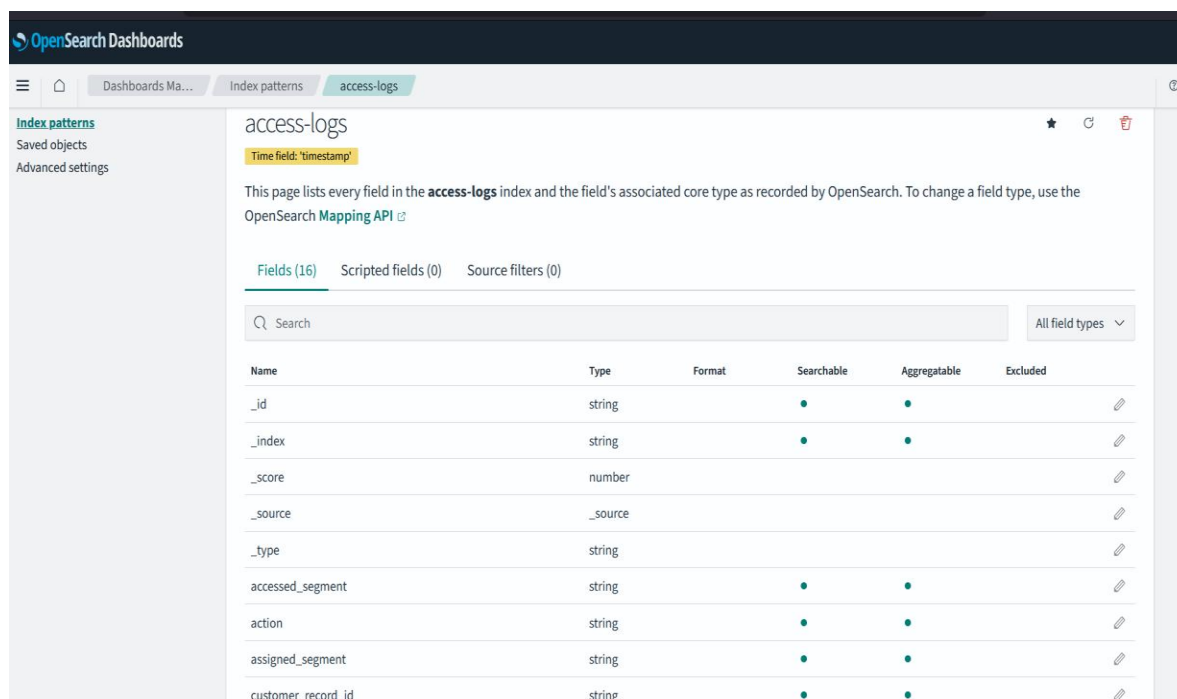


Figure 5.7a — *index_pattern_access-logs.png*: index pattern configuration for access-logs.

OpenSearch Dashboards

Index patterns alerts*

Time field: 'timestamp' Default

This page lists every field in the **alerts*** index and the field's associated core type as recorded by OpenSearch. To change a field type, use the OpenSearch [Mapping API](#).

Fields (12) Scripted fields (0) Source filters (0)

Search

Name	Type	Format	Searchable	Aggregatable	Excluded
_id	string		•	•	
_index	string		•	•	
_score	number				
_source	_source				
_type	string				
alert_id	string		•	•	
employee_id	string		•	•	
event_id	string		•	•	
reasons	string		•	•	

Figure 5.7b — index_pattern_alerts.png: index pattern configuration for alerts.

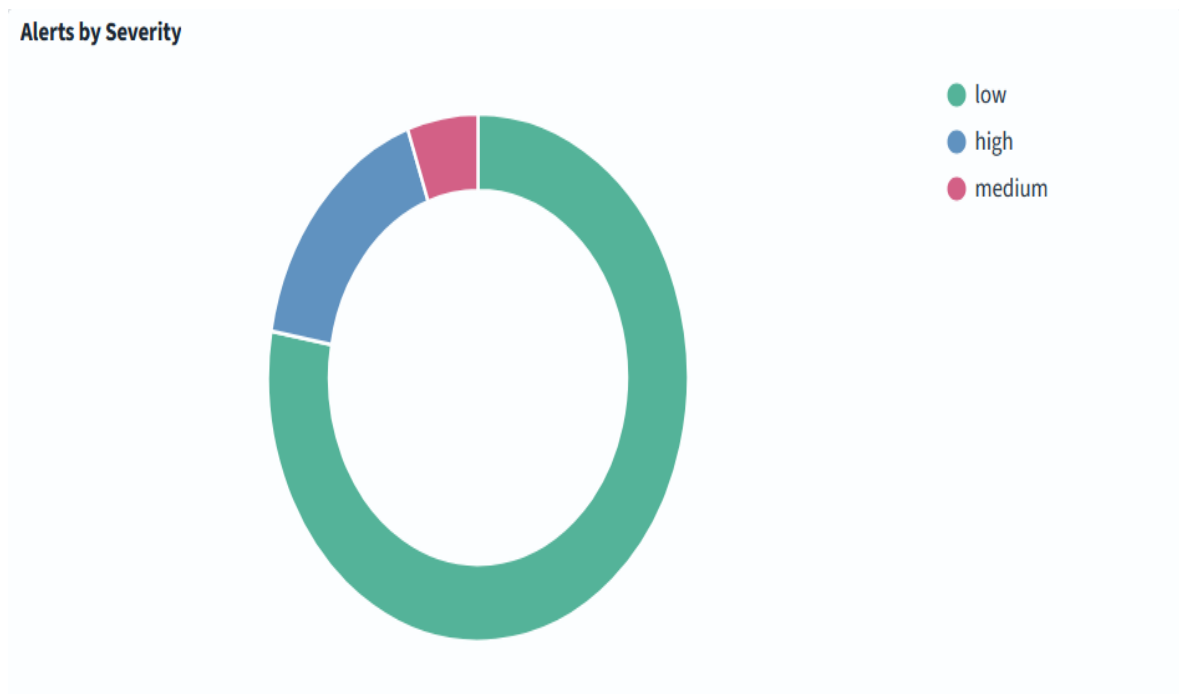


Figure 5.7c — alerts_by_severity.png: Alerts by Severity visualization.

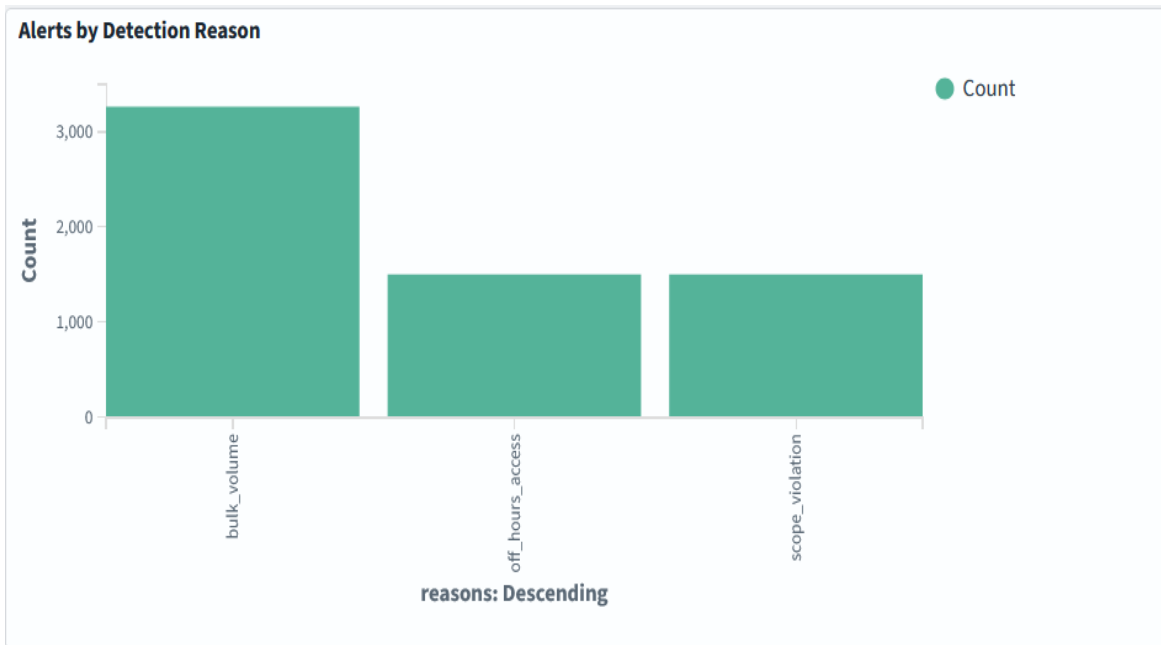


Figure 5.7d — alerts_by_reason.png: Alerts by Detection Reason visualization.

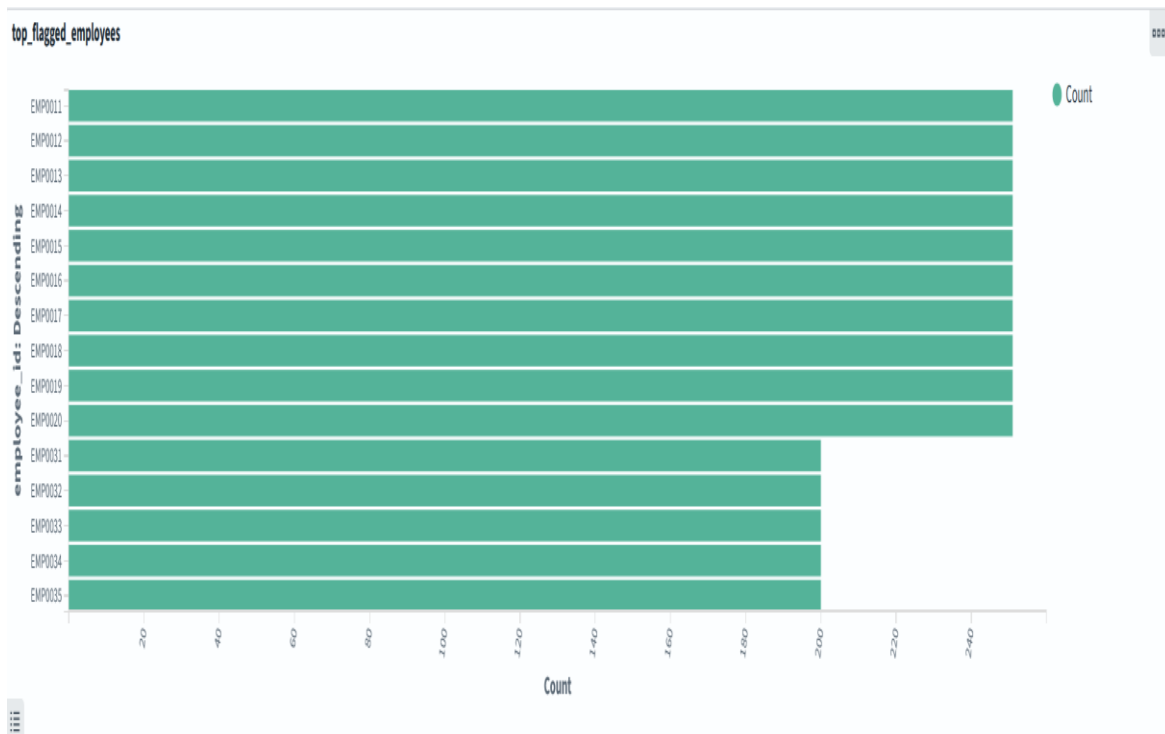


Figure 5.7e — top_flagged_employees.png: Top Flagged Employees visualization.

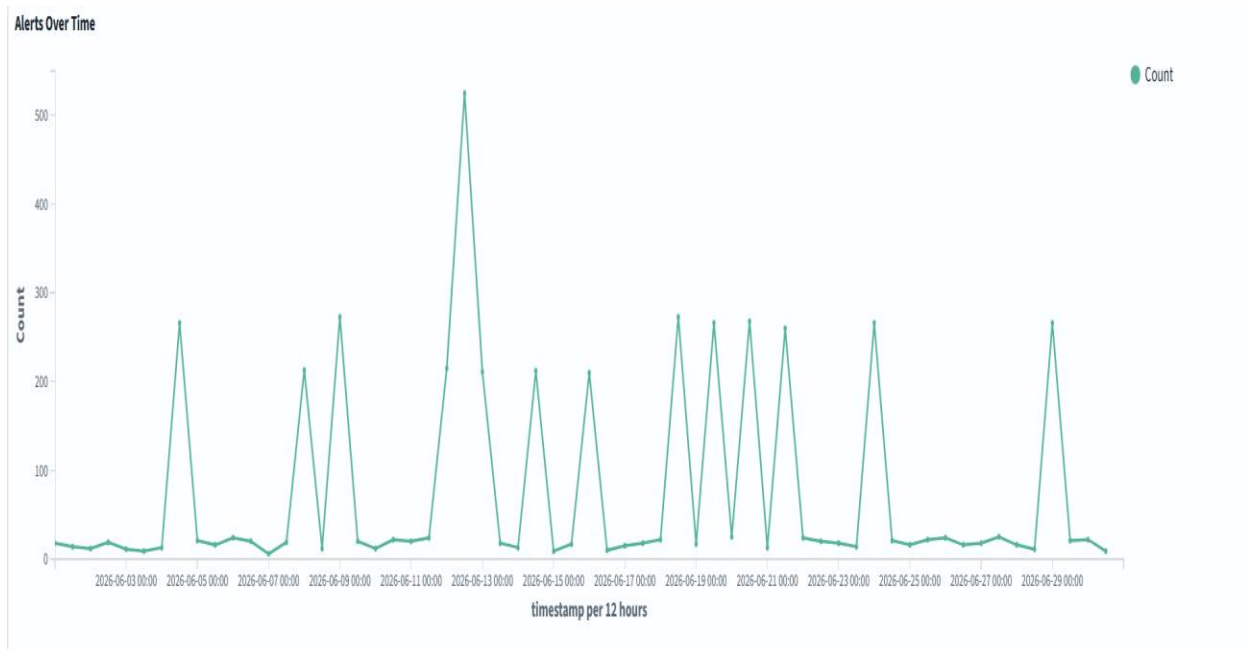


Figure 5.7f— Alerts Over Time .png: Alerts Over Time visualization.

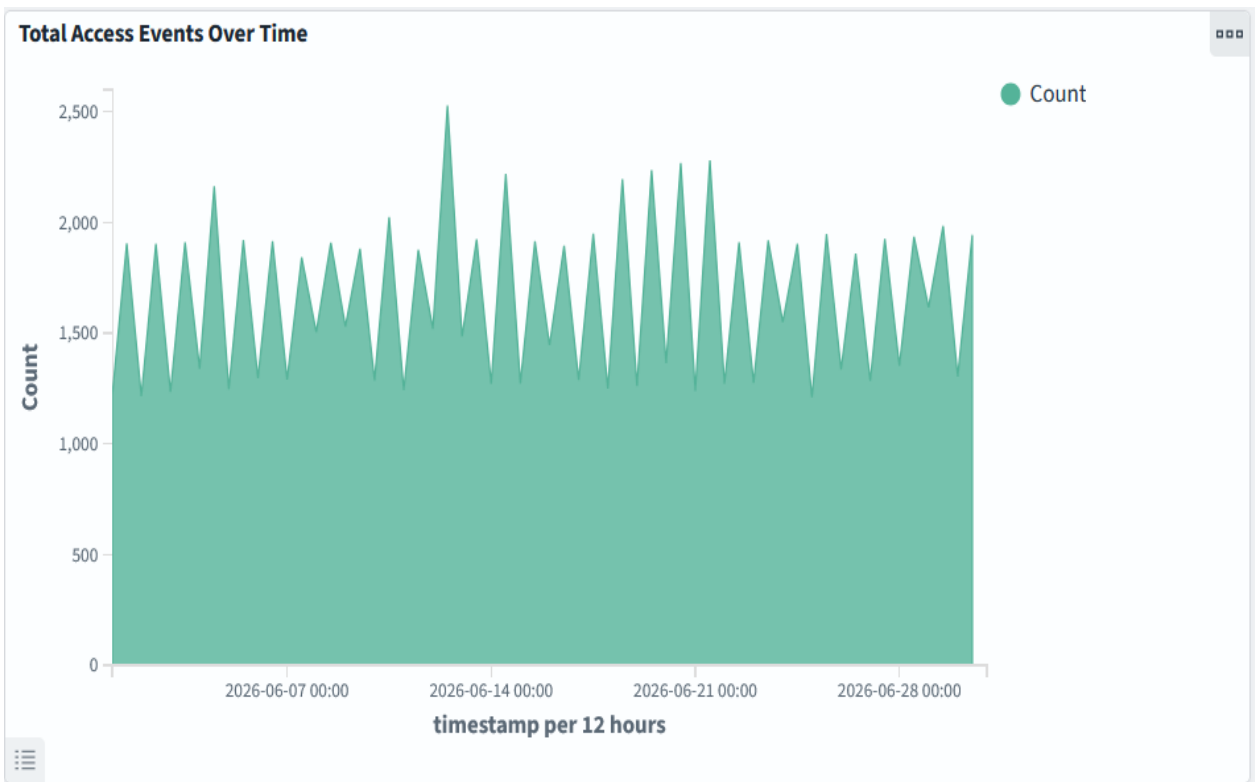


Figure 5.7g — total_access_events_over_time.png: Total Access Events Over Time visualization.

Events by Action Type

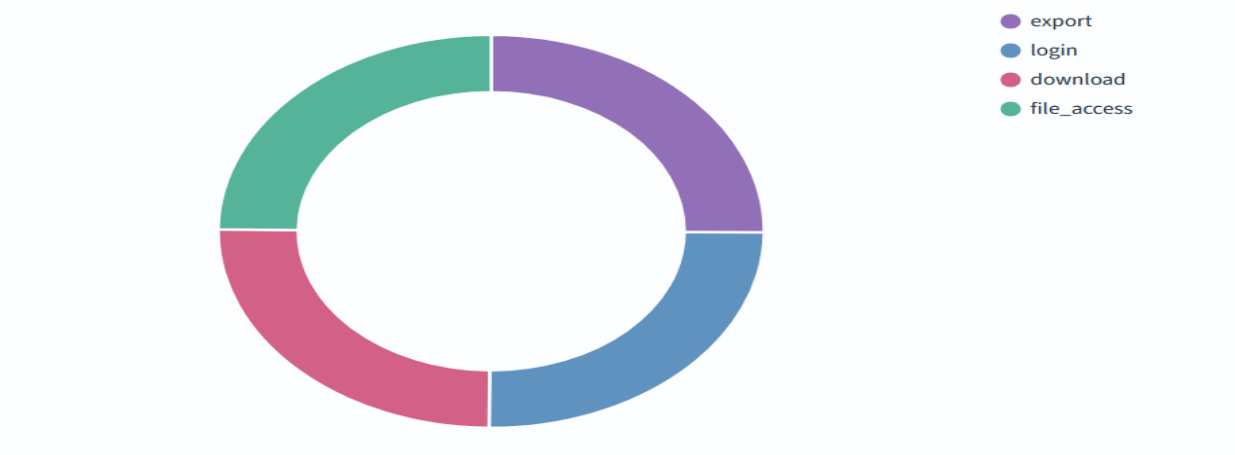


Figure 5.7h — events_by_action_type.png: Events by Action Type visualization.

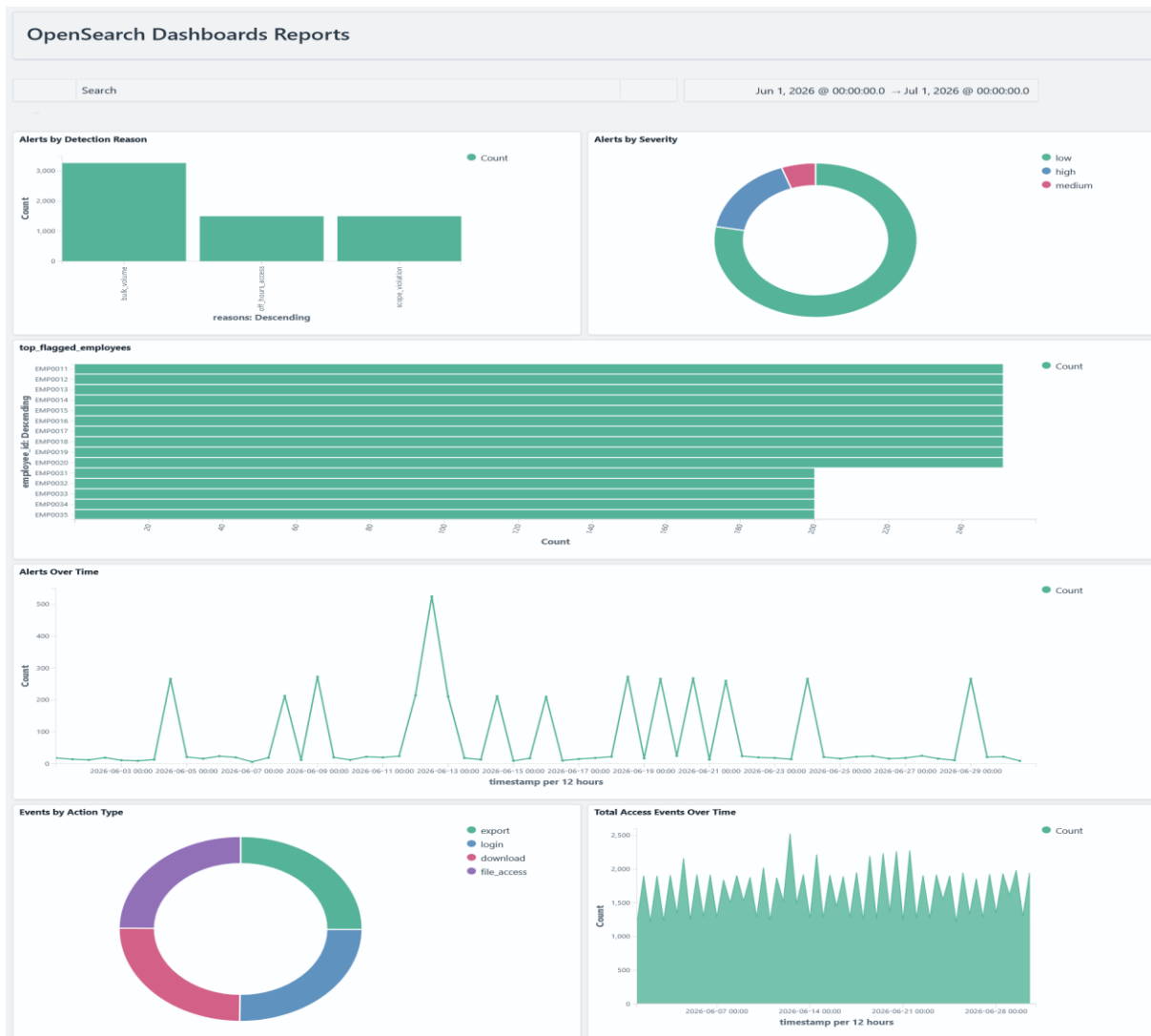


Figure 5.7i — full_dashboard_report.png: full combined "Insider Threat Detection Dashboard" export, all six panels.

5.8 07_testing — Security Testing

Filename	Description
test_matrix.py	Automated script running all 7 tests from the Security Testing Matrix (Section 4).
test_matrix_output.txt	Terminal output of the final test run — all 7 tests PASS.
investigate_fp_fn.py	Root-cause investigation script for false positives/negatives (Section 4.1).
investigate_fp_fn_output.txt	Terminal output of the root-cause investigation, confirming the forward-window explanation.

```

=====
STEP 7 - SECURITY TESTING MATRIX
=====

[PASS] Test 1: Normal behavior -> no alert
  Expected: 0 alerts for EMP0100 (normal employee, standard hours, correct segment)
  Actual:   0 alert(s) found for EMP0100

[PASS] Test 2: Off-hours access -> alert fires
  Expected: Alert(s) with reason 'off_hours_access' for EMP0001
  Actual:   50 off-hours alert(s) found out of 50 total alerts for EMP0001

[PASS] Test 3: Bulk volume -> alert fires
  Expected: Alert(s) with reason 'bulk_volume' for EMP0011
  Actual:   251 bulk-volume alert(s) found out of 251 total alerts for EMP0011

[PASS] Test 4: Out-of-segment access -> alert fires
  Expected: Alert(s) with reason 'scope_violation' for EMP0021
  Actual:   50 scope-violation alert(s) found out of 50 total alerts for EMP0021

[PASS] Test 5: Multi-signal -> higher severity score
  Expected: Some alerts for EMP0031 reach 'high' severity (3 signals combined)
  Actual:   Severity breakdown for EMP0031: {'high': 151, 'medium': 49}

[PASS] Test 6: Alert data completeness
  Expected: All required fields present: ['alert_id', 'employee_id', 'event_id', 'timestamp', 'reasons', 'severity']
  Actual:   Sample alert record: {'alert_id': 'ALERT0000001', 'employee_id': 'EMP0028', 'event_id': 'EVT0099854', 'timestamp':
'2026-06-01 10:46:41+0300', 'reasons': ['scope_violation'], 'severity': 'low', 'signal_count': 1} | All fields present

Running Test 7 - this scans all 100,000 records, please wait...
[PASS] Test 7: Precision/Recall against full dataset
  Expected: Recall >= 90% and Precision >= 95% against is_suspicious ground truth
  Actual:   Precision: 100.00% (4510 TP / 4510 total alerts) | Recall: 90.20% (4510 TP / 5000 planted-suspicious) | False posit
ives: 0 | False negatives (missed): 490
=====

```

Figure 5.8a — test_matrix_screenshot.png: terminal output showing all 7 tests passing.

N.B : All configuration files, scripts, datasets, and log files referenced in this report are included as supporting evidence, consistent with the assignment's evidentiary requirements.

6. Risk Assessment Table

The following risks were identified through our assessment and testing process, along with their estimated impact, likelihood, and recommended controls.

Identified Risk	Impact	Likelihood	Recommended Control
Undetected insider threats near burst tail-ends (false negatives)	High	Medium	Implement a bidirectional detection window (counting events both before and after) instead of forward-only; lower the bulk-volume threshold with continued false-positive monitoring.
Alert fatigue from excessive false alarms	Medium	Medium	Current system already achieves 100% precision; Incorporate contextual information such as approved overtime, authorised bulk operations, scheduled maintenance windows, and documented business exceptions to reduce false-positive alerts for production.
Detection system validated only on synthetic data	Medium	Medium	Pilot the system against a sample of real (anonymized) access logs before full production deployment; treat current metrics as a lower bound on real-world performance.
No authentication enabled on the OpenSearch cluster itself	High	Medium	Enable the OpenSearch Security plugin with TLS and role-based cluster access before any production use; the current development setup runs without cluster authentication.
No encryption at rest or in transit for stored logs	High	Medium	Enable OpenSearch's built-in encryption at rest and enforce HTTPS for all client-to-cluster traffic in a production deployment.
Single-node OpenSearch deployment (no redundancy)	Medium	Low	Move to a multi-node OpenSearch cluster with replica shards, and implement scheduled backups (snapshots) of both indices.
No Employee role and IP-familiarity check	Low	Medium	Extend detection logic with a fourth signal checking access from IPs outside an employee's known set, and role-specific access rules.
Detection operates in batch mode, not real time	Medium	Medium	Introduce streaming ingestion so detection runs continuously rather than on a scheduled/manual basis.

7. Conclusion and Recommendations

7.1 Vulnerabilities Identified

- As a single-node deployment without redundancy.
- The OpenSearch cluster used in this project runs without authentication or TLS enabled, appropriate for an isolated development environment but not for production use.

- The system has not yet been validated against real, non-synthetic access logs.
- The current bulk-volume detection algorithm uses a forward-looking rolling window, which can miss suspicious events occurring near the end of a sustained high-volume access burst because insufficient subsequent events remain within the analysis window.
- The detection engine evaluates behavioural rules without considering operational context, such as approved overtime, authorised bulk operations, scheduled maintenance activities, or documented business exceptions.
- The detection engine focuses primarily on behavioural indicators such as access timing, access volume, and customer-segment violations. Other measures like IP-based anomaly detection and role-specific access policies are not currently evaluated.

7.2 Remaining Risks

- A failure of the single OpenSearch node could result in temporary loss of monitoring capability and delayed investigation of security incidents.
- Because the lab project focuses on insider threats involving authenticated employees, insufficient authentication and access controls on the OpenSearch platform could allow a privileged insider to tamper with or access security logs, reducing the integrity and reliability of audit records at production.
- Real-world detection performance may differ from the results reported here.
- Approximately 9.8% of genuinely suspicious behavior (490 of 5,000 planted events) would not generate an alert under the current detection thresholds, concentrated specifically at the tail end of high-volume access bursts.
- Legitimate business activities, such as authorised overtime work or approved large-scale data exports, may trigger alerts because the detection rules do not currently incorporate contextual information such as approved exceptions or scheduled maintenance activities.
- Insider activities involving unfamiliar IP addresses or unauthorized role-based access may remain undetected because these indicators are not currently evaluated by the detection engine.

7.3 Lessons Learned

- A rule-based detector is only as good as the ground truth used to validate it: an early version of our dataset generator produced ambiguous labels at the boundary between planted bursts and random normal activity, which taught us to treat ground-truth construction itself as something requiring careful design and verification, not just detection logic.
- Investigating anomalies numerically (through backward/forward window counts) rather than assuming a cause proved essential; our initial explanation for the false negatives was directionally correct but needed direct evidence before it could be reported with confidence.
- Precision and recall involve a genuine trade-off, not a simple 'higher is always better' metric; our team learned to reason about which of the two mattered more for this specific scenario (an insider-threat team that cannot afford constant false alarms) rather than optimizing blindly.
- Fixing a subtle timezone bug (an incorrect historical UTC offset from a Python library default) early in the project reinforced the importance of validating even seemingly minor implementation details before building further logic on top of them.

7.4 Improvements That Should Be Implemented

- Redesign the bulk-volume rule to use a bidirectional (centered) rolling window, which our investigation confirmed would capture the majority of currently-missed tail events without requiring a lower, noisier threshold.
- Incorporate IP-familiarity and role-based rules as additional detection signals, using data already present in the access logs but not yet consumed by the detection logic.

- Enable the OpenSearch Security plugin (authentication, TLS, and cluster-level role-based access control) before any deployment beyond a development environment.
- Move to a multi-node OpenSearch cluster with scheduled snapshots, removing the single-node availability risk.
- Pilot the system against a sample of real, anonymized access logs to validate that detection performance holds outside of the synthetic dataset used for this project.
- As a longer-term enhancement, consider training a supervised machine-learning classifier using the same feature set (timing, segment match, access frequency) once sufficient analyst-confirmed real-world labels are available, allowing detection thresholds to be learned rather than manually defined.
- Incorporate contextual information such as approved overtime, maintenance windows, and authorized bulk operations to reduce unnecessary alerts.

Overall, this project demonstrates a working, thoroughly tested insider-threat detection capability built entirely on open-source tooling, achieving 100% precision and 90.20% recall against a rigorously validated synthetic dataset. The identified limitations are specific, well-understood, and directly actionable, providing the client with both a functioning system today and a clear, evidence-based roadmap for further improvement.

8. Appendices

Appendix A: Scripts Included

- `generate_data.py` : synthetic dataset generator
- `load_to_opensearch.py` : bulk loader into OpenSearch
- `detect_threats.py` : detection logic and alert generation
- `verify_alerts.py` : ground-truth validation of alert accuracy
- `test_matrix.py` : automated 7-test security testing suite
- `investigate_fp_fn.py` : root-cause investigation of false positives/negatives
- `top_flagged_employees_table.py` : summary table generator for reporting

Appendix B: Dataset Schema

Each record in `access_logs.csv` / the `access-logs` index contains: `event_id`, `employee_id`, `employee_role`, `assigned_segment`, `timestamp`, `action`, `customer_record_id`, `accessed_segment`, `source_ip`, `session_duration_minutes`, and `is_suspicious` (ground-truth label used only for testing, never consumed by detection logic).

Each record in the `alerts` index contains: `alert_id`, `employee_id`, `event_id`, `timestamp`, `reasons` (list), `severity`, and `signal_count`.

Appendix C: Tool Versions and References

- OpenSearch Documentation. <https://opensearch.org/docs/latest/>
- OpenSearch Dashboards Documentation. <https://opensearch.org/docs/latest/dashboards/>
- Docker Documentation. <https://docs.docker.com/>
- Python Documentation. <https://docs.python.org/3/>
- opensearch-py Client Documentation. <https://opensearch-project.github.io/opensearch-py/>
- DSA 4030 End of Semester Practical Group Project Instruction Document